

ENGR 102 Summer Bridge

Day 12 Instructor Key

For Loops, Range Bounds, Counters, and Accumulators

20 points • approximately 20-25 minutes

Name		UIN		Section	
------	--	-----	--	---------	--

Boolean-operator convention. Python operators appear as [and](#), [or](#), and [not](#). Their lowercase spelling is required in code.

Show intermediate state for trace credit. Program credit requires one coherent solution. Unless a prompt says otherwise, give exact Python 3 output.

Question 1. Trace every range-driven state update.

5 points

```
total = 0
count = 0

for number in range(2, 10, 2):
    total = total + number
    if total >= 10:
        count = count + 1
    print(number, total, count)
```

List range values, then give number, total, and count after every pass and all exact output.

Answer and explanation

range produces 2, 4, 6, 8. States and output are 2 2 0; 4 6 0; 6 12 1; 8 20 2. The counter tests the updated running total on each pass.

Question 2. Repair an excluded-stop defect.

3 points

```
limit = 4
total = 0

for number in range(1, limit):
    total = total + number

print(total)
```

The intent is 1 through limit inclusive. Give actual, intended, cause, repair, and one smallest boundary test.

Answer and explanation

The actual total is $1 + 2 + 3 = 6$; the intended total is 10. range excludes its stop, so repair with `range(1, limit + 1)`. For limit 1, the original loop gives 0 while the repaired loop gives 1.

Question 3. Program construction**12 points**

Write one complete Python program that satisfies every requirement.

- Read `sample_count` as an integer and `threshold` as a float.
- If `sample_count` is not positive, print invalid and request no readings.
- Otherwise read exactly `sample_count` floating-point readings.
- Compute total, average, and the count of readings at or above threshold.
- Print Total to one decimal, Average to two decimals, and At or above as an integer on separate labeled lines.

Answer and explanation

```
sample_count = int(input())
threshold = float(input())

if sample_count <= 0:
    print("invalid")
else:
    total = 0.0
    at_or_above = 0
    for sample in range(sample_count):
        reading = float(input())
        total = total + reading
        if reading >= threshold:
            at_or_above = at_or_above + 1
    average = total / sample_count
    print(f"Total: {total:.1f}")
    print(f"Average: {average:.2f}")
    print(f"At or above: {at_or_above}")
```

The positive-count guard prevents invalid repetition and division by zero. Both summary states are initialized once before the loop; average is computed after all readings are processed.

For count 3, threshold 4, and readings 3.5, 4, 5, output is Total: 12.5, Average: 4.17, and At or above: 2. Equality at the threshold is counted.

Scoring guidance

2 guard/input, 2 initialization, 3 exact loop/repeated input, 2 total/count logic, 1 average/output, 2 boundary test.